

FAME: Few-Shot Adaptive Perturbation Generation for Scalable Website Fingerprinting Defense via Mixture-of-Experts

Abstract—Tor anonymous communication remains vulnerable to Website Fingerprinting (WF) attacks that infer visited websites from encrypted traffic patterns. Recent adversarial defenses generate optimized perturbations on burst sequences to mislead deep-learning classifiers under low bandwidth overhead. However, these defenses are designed for fixed experimental settings and face three key limitations in dynamic real-world environments: (1) per-class modeling that scales poorly to large, expanding monitored sets; (2) inability to adapt to newly emerging website classes without costly retraining; and (3) concentrated dummy-packet injection that creates detectable local traffic anomalies. We propose FAME, a few-shot adaptive perturbation generation framework based on the Mixture-of-Experts (MoE) architecture. FAME maintains a shared expert generator pool coupled with a feature-aware gating router, decoupling perturbation capacity from the number of website classes. For classes unseen during training, a routing-prototype induction mechanism aggregates a small support set into a class-level routing prototype and derives defense rules without any parameter update. A perturbation-dispersion loss further suppresses localized injection, improving the statistical naturalness of defended traces. Experiments on the AWF and DF datasets against three deep-learning attacks (AWF, DF, VarCNN) demonstrate that FAME achieves the lowest attack accuracy in the known-class closed-world setting while maintaining bandwidth overhead around 5%. In the few-shot adaptation setting, FAME substantially outperforms both rule-based and adversarial baselines on unseen classes. Perturbation analysis confirms more spatially dispersed dummy-packet distributions, and open-world evaluation shows that defended traffic is nearly indistinguishable from unmonitored traffic (AUC ≈ 0.5). Deployment prototypes on a Tor Pluggable Transport (PT) plugin and a Tofino P4 switch confirm negligible impact on page loading time and approximately 300 ns additional per-packet latency.

I. INTRODUCTION

Tor [1] is a widely deployed anonymous communication system that conceals communication relationships by forwarding encrypted traffic through multiple relays. Despite its encryption, Tor remains vulnerable to Website Fingerprinting (WF) attacks: a passive adversary observing encrypted traffic between the user and the Tor entry node can infer the visited website from side-channel features such as packet directions, timing patterns, and burst structures. Deep-learning-based at-

tacks [2], [3], [4], [5], [6] automatically extract discriminative patterns from the widely used *direction sequence* representation (Fig. 1(a)), achieving over 98% accuracy in closed-world settings.

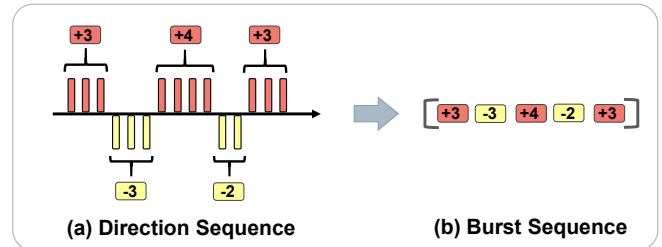


Fig. 1. Direction sequence and burst sequence representations.

To counter WF attacks, existing defenses inject dummy packets, delay transmissions, or reshape traffic to obscure website-specific patterns. Traditional padding and traffic-shaping approaches [7], [8], [9], [10], however, often incur substantial bandwidth or latency overhead while offering limited protection against adaptive deep-learning attackers. Adversarial-example-based defenses [10], [11], [12], [13], [14] address this gap by generating optimized perturbations that mislead classifiers with lower overhead. Because inserting dummy packets into direction sequences changes both the sequence length and local structure—rendering the optimization space discrete and non-continuous [15]—recent adversarial defenses increasingly operate on *burst sequences*, which aggregate consecutive same-direction packets into signed burst lengths [14], [13], [16]. As shown in Fig. 1(b), dummy insertion under this representation only modifies burst magnitudes, enabling stable differentiable optimization.

Despite these advances, burst-sequence-based adversarial defenses remain designed for fixed experimental settings and face three key limitations when deployed in dynamic real-world environments.

Limited scalability. Existing adversarial defenses rely on class-specific generation paradigms, maintaining an independent generator or perturbation template for each monitored website. As the monitored set grows—a common occurrence in real-world deployments—training cost, storage overhead, and rule-management complexity scale linearly with the number of classes, hindering large-scale deployment.

Limited adaptability to emerging classes. Practical anonymous communication is not a one-time optimization over a

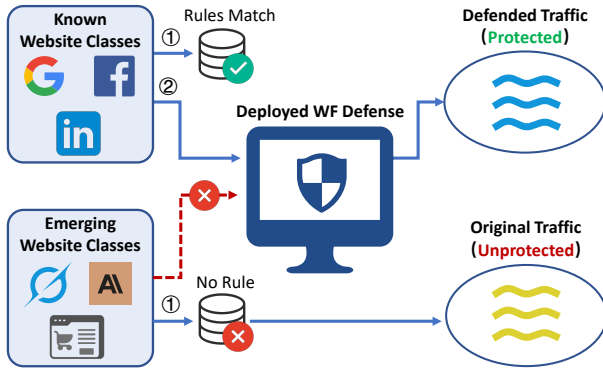


Fig. 2. New website classes without corresponding defense rules.

static website set. New websites continuously appear, existing pages update their content, and Tor browser or network-path changes cause traffic distribution shifts [17], [18]. As illustrated in Fig. 2, when a previously unobserved class appears after deployment, the defense system lacks a corresponding perturbation rule, leaving traffic unprotected. Recollecting samples, retraining models, and redeploying rules incurs substantial maintenance overhead and, critically, creates a *protection gap* until updated rules become available.

Insufficient perturbation stealthiness. Existing adversarial defenses tend to concentrate dummy packets on a few gradient-sensitive bursts, creating conspicuous local traffic spikes. While effective at reducing classification accuracy, such concentrated perturbations may disrupt the statistical naturalness of defended traffic and increase the risk of being exposed by anomaly-based inspection. A practical defense should reduce attack accuracy without introducing obvious perturbation patterns.

To address all three limitations, we propose **FAME**, a **F**ew-shot **A**ddaptive **M**ixture-of-**E**xperts based perturbation generation framework. The design philosophy maps each limitation to a dedicated architectural component:

(i) *Scalability via shared experts.* We construct a shared expert generator pool coupled with a feature-aware gating router, enabling both known and unseen websites to share a unified perturbation-generation space. This decouples model capacity from the number of website classes, eliminating linear parameter growth.

(ii) *Adaptability via routing-prototype induction.* For a newly emerging class, we aggregate the expert-routing vectors of a small support set into a class-level *routing prototype* and directly derive the defense rule—without any parameter update or retraining—reducing adaptation latency to the time needed for sample collection and a single forward pass.

(iii) *Stealthiness via dispersion regularization.* A perturbation-dispersion loss penalizes per-burst injection exceeding a threshold, encouraging dummy packets to spread across the burst sequence and preserving the statistical naturalness of defended traces.

FAME operates in two stages. *Offline*, the shared expert pool and gating router are jointly trained on traffic from known classes. *Online*, known-class traffic is routed through the gat-

ing module to generate class-differentiated perturbations; when an unseen class arrives, the routing-prototype mechanism rapidly induces an effective defense rule at negligible cost. This paradigm shifts WF defense from per-class independent modeling to *one-time learning with continual adaptation*.

We conduct extensive experiments spanning known-class closed-world defense (Sec. V-D), few-shot adaptation to unseen classes (Sec. V-E), perturbation dispersion analysis (Sec. V-F), open-world evaluation (Sec. V-H), and hardware deployment (Sec. V-I). Results demonstrate that FAME achieves the lowest attack accuracy with $\sim 5\%$ bandwidth overhead, adapts effectively to unseen classes with as few as five support samples, produces more spatially dispersed perturbations, renders defended traffic nearly indistinguishable from unmonitored traffic ($AUC \approx 0.5$), and introduces negligible latency overhead on both a Tor PT plugin and a Tofino P4 switch.

Contributions.

- 1) **Scalable MoE-based perturbation generation.** We are the first to address the scalability bottleneck of per-class generators in WF defenses. A shared expert pool with feature-aware gating router adaptively combines perturbation patterns without per-class generators, scaling to large and expanding monitored sets [19].
- 2) **Few-shot routing-prototype induction.** We are the first to address the defense problem for newly emerging websites. For classes unseen during training, FAME aggregates expert-routing vectors from a small support set into a class-level prototype to derive static defense rules—requiring no gradient updates and completing in milliseconds.
- 3) **Dispersion-aware perturbation design.** A perturbation-dispersion loss suppresses concentrated injection, yielding less locally concentrated defended traces under strict bandwidth constraints.
- 4) **Comprehensive evaluation and practical deployment.** We evaluate FAME under known-class, emerging-class, and open-world settings, and validate deployability via a Tor PT plugin and a P4 switch data-plane implementation with negligible latency overhead.

II. BACKGROUND AND PROBLEM DEFINITION

A. Threat Model

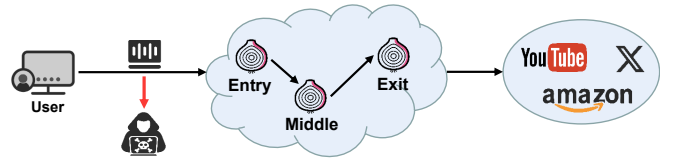


Fig. 3. Threat model of Website Fingerprinting attacks.

We adopt a standard WF threat model consistent with prior work [13], [10] (Fig. 3). Users browse the web through the Tor browser to conceal their identities and browsing targets.

The adversary operates in three phases. First, during an *offline training phase*, the adversary actively visits a set of

monitored websites via the Tor network and collects direction-based packet traces to train a deep-learning classifier. Second, during the *observation phase*, the adversary passively monitors the victim's traffic between the Tor client and the entry node, observing only packet-level metadata (direction and timing) without access to payload content. The attacker is non-invasive: it cannot decrypt, modify, inject, or drop real packets. Third, during the *inference phase*, the observed trace is fed into the trained classifier to infer the visited website.

To counter this threat, WF defenses are deployed as *TOR PT* at the client/bridge or offloaded to *programmable data planes* (e.g., P4 switches). By injecting dummy packets, these defenses obfuscate flow characteristics and degrade classifier accuracy. This paper focuses on *insert-only* defenses: the defender may only add dummy packets to existing bursts and cannot delete, delay, or reorder real packets.

B. Burst Sequence Representation

Existing WF studies commonly represent traffic traces as *direction sequences* (DS), as shown in Fig. 1(a). For the i -th trace:

$$DS^i = \langle d_i^1, d_i^2, \dots, d_i^n \rangle, \quad d_i^j \in \{+1, -1\}, \quad (1)$$

where n is the number of packets, $+1$ denotes the client-to-server (upstream) direction, and -1 the server-to-client (downstream) direction.

DS are inherently discrete and variable-length; inserting dummy packets changes both the length and local structure, making gradient-based optimization intractable [15]. To address this, recent work [14], [13] adopts the *burst sequence* (BS) representation, which aggregates consecutive same-direction packets into a single burst (Fig. 1(b)):

$$BS^i = \langle |b_i^1| \cdot d_i^1, |b_i^2| \cdot d_i^2, \dots, |b_i^m| \cdot d_i^m \rangle, \quad (2)$$

where m is the number of bursts and $|b_i^j|$ is the packet count of the j -th burst. For example, $DS^i = \langle +1, +1, -1, -1, -1, +1, +1, +1, -1, -1 \rangle$ yields $BS^i = \langle +2, -3, +3, -2 \rangle$.

Under this representation, inserting dummy packets modifies only the magnitude of affected bursts while preserving sequence length and directional structure. Burst sequences are therefore amenable to continuous perturbation optimization and translate directly into deployable rules specifying the number of dummy packets to insert per burst. We adopt burst sequences throughout this paper.

C. Burst-Level Defense Objective

Let bs denote a burst sequence and $f(\cdot)$ the attacker's classifier, producing a predicted class $\hat{y} = f(bs)$. The defender generates a non-negative perturbation vector δ to produce the defended trace:

$$\tilde{bs} = bs + \delta, \quad \delta \geq 0. \quad (3)$$

Each element of δ specifies the number of dummy packets inserted at the corresponding burst position. In the *known-class* setting, the defense objective is:

$$\min_{\delta_y} \|\delta_y\|_1, \quad \text{s.t.} \quad \forall bs \in BS_y, \quad f(bs + \delta_y) \neq y. \quad (4)$$

This formulation captures the basic goal of fixed-class defenses. However, real-world deployments must also handle classes not observed during training, motivating the dynamic adaptation setting below.

D. Dynamic Few-Shot Adaptation Setting

In practice, the monitored website set is neither fixed nor static. New websites emerge, page content evolves, and network conditions shift. If the defense must collect large sample sets and fully retrain after every such change, the resulting cost and rule-update latency become prohibitive. We therefore formalize a *dynamic few-shot defense setting* that bridges Eq. 4 to realistic deployment conditions.

Let the classes known during training be $\mathcal{Y}_{base} = \{1, 2, \dots, C\}$. After deployment, novel classes appear such that:

$$\mathcal{Y}_{novel} \cap \mathcal{Y}_{base} = \emptyset. \quad (5)$$

For any $y' \in \mathcal{Y}_{novel}$, the defender has access only to a small support set:

$$S_{y'} = \{bs_1, bs_2, \dots, bs_{n_s}\}, \quad (6)$$

where n_s is much smaller than the sample size used for full training. After the defense rule is induced, it is applied to subsequent query traces from the same novel class, denoted as $Q_{y'}$, which are not used for rule induction. The objective is to induce a defense rule $\delta_{y'}$ from $S_{y'}$ alone, *without updating any trained parameters*, so that query traces are misclassified after defense:

$$f(bs + \delta_{y'}) \neq y', \quad bs \in Q_{y'}. \quad (7)$$

The problem studied in this paper is therefore: **how to learn a burst-level perturbation generation mechanism that covers large-scale known classes and rapidly adapts to newly emerging classes using only a few support samples, under a strict bandwidth budget.** We require the mechanism to satisfy four properties:

- 1) **High capacity.** The model must capture complex, class-differentiated traffic patterns without per-class generators.
- 2) **Few-shot adaptability.** Usable defense rules should be derivable from a handful of support samples without retraining.
- 3) **Low overhead.** Perturbation magnitude must be minimized to keep bandwidth overhead small while maintaining strong misdirection.
- 4) **Statistical stealthiness.** Perturbations should be spatially dispersed across burst positions to avoid conspicuous local anomalies.

III. METHOD DESIGN

A. Framework Overview

Fig. 4 illustrates the FAME framework. The system generates effective, low-overhead, and statistically natural burst-level defense rules for both known and unseen website classes through three stages, each targeting a specific requirement from Sec. II-D.

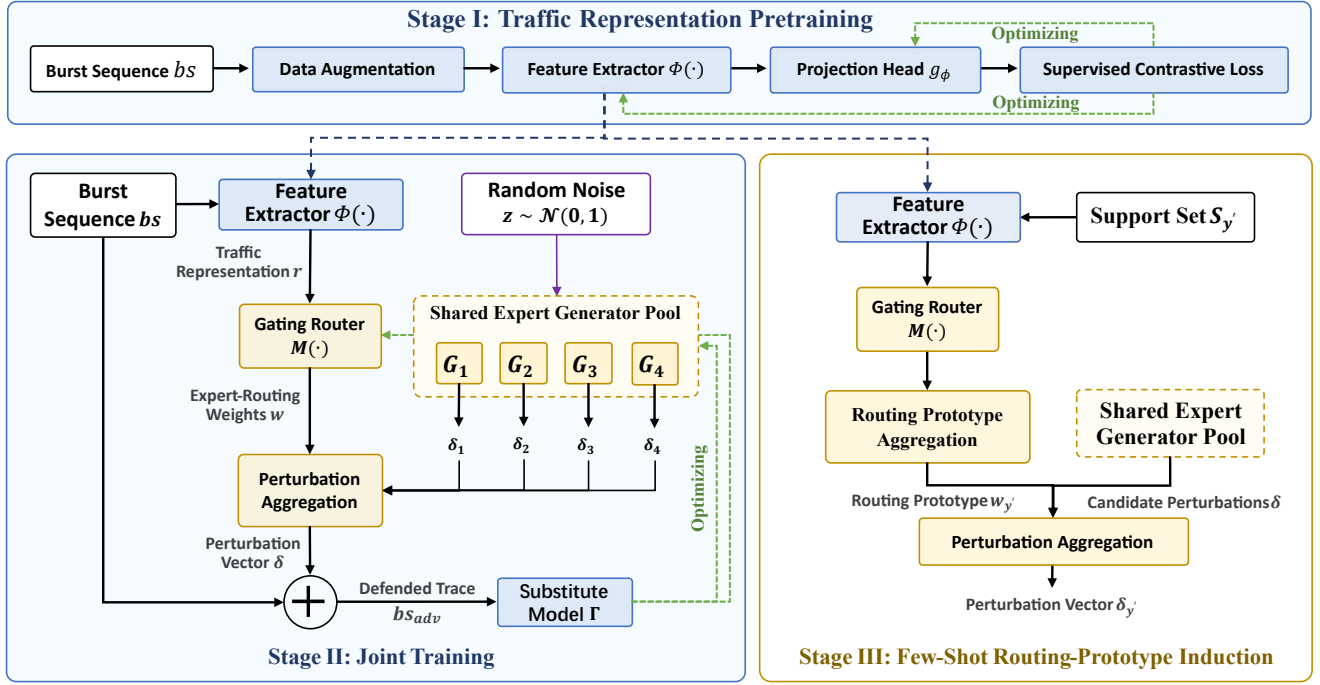


Fig. 4. Overview of the FAME framework. The three stages are: (I) traffic representation pretraining via supervised contrastive learning; (II) joint training of the feature-aware gating network and the shared expert generator pool; and (III) few-shot routing-prototype induction for newly emerging website classes.

Stage I: Traffic representation pretraining (*supports few-shot adaptability*). Raw burst sequences are high-dimensional and sensitive to network fluctuations, making few-shot estimation unreliable on raw inputs. We learn a feature extractor $\Phi(\cdot)$ that maps each burst sequence to a compact, class-discriminative representation:

$$r = \Phi(bs), \quad (8)$$

$\Phi(\cdot)$ is trained with supervised contrastive learning to pull same-class representations together and push different-class representations apart. The resulting feature space ensures that even a few support samples from an unseen class yield geometrically stable representations, enabling reliable prototype estimation in Stage III.

Stage II: Feature-aware expert routing training (*provides high capacity*). We construct a Mixture-of-Experts perturbation generation model comprising a shared expert generator pool

$$\mathcal{G} = \{G_1, G_2, \dots, G_K\}, \quad (9)$$

and a gating network. Each expert takes random noise z and produces a candidate perturbation vector; the gating network converts r into expert-routing weights that determine each expert's contribution. Because capacity stems from *shared* expert combinations rather than per-class generators, the model scales to large class sets without parameter explosion.

Stage III: Few-shot routing-prototype induction (*enables rapid adaptation*). For an unseen class, we compute the expert-routing vectors of a small support set and average them to obtain a class-level *routing prototype*. This prototype, combined with the shared expert pool, directly yields a static defense rule. The process requires only forward inference and

mean aggregation—no gradient updates—minimizing adaptation cost to sample collection and a single inference pass.

The key insight enabling Stage III is that the contrastive feature space (Stage I) clusters same-class samples tightly, so their gating vectors concentrate in a small region of the expert-combination space. Averaging these vectors yields a representative centroid even from a handful of samples—analogue to prototype networks in few-shot learning [20], but applied to the expert-routing space rather than a classification embedding.

B. Traffic Representation Learning

This subsection details Stage I, whose goal is to provide a stable, class-discriminative feature space supporting both the gating network (Stage II) and few-shot prototype aggregation (Stage III), thereby enabling support for unseen samples without requiring model updates.

Given a batch $\{(bs_i, y_i)\}_{i=1}^B$, the feature extractor produces low-dimensional representations $r_i = \Phi(bs_i)$. We train $\Phi(\cdot)$ with supervised contrastive learning [21]. The positive set for sample i contains all other samples sharing the same label:

$$P(i) = \{p \mid y_p = y_i, p \neq i\}. \quad (10)$$

The supervised contrastive loss is:

$$L_{\text{sup}} = \sum_{i=1}^B \frac{-1}{|P(i)|} \sum_{p \in P(i)} \log \frac{\exp(\text{sim}(r_i, r_p)/\tau_c)}{\sum_{a \neq i} \exp(\text{sim}(r_i, r_a)/\tau_c)}, \quad (11)$$

where $\text{sim}(\cdot, \cdot)$ is cosine similarity and τ_c is a temperature hyperparameter, set to 0.1 in our implementation. This objective shapes inter-sample distances such that a small number of support samples from a novel class suffice to capture its

traffic structure without being dominated by per-sample noise. After pretraining, $\Phi(\cdot)$ is frozen and reused as a shared encoder throughout subsequent stages.

This contrastive feature space ensures that even a few support samples from an unseen class produce a stable representation centroid, directly enabling the few-shot adaptability required for emerging websites (addressing the adaptability limitation).

C. Feature-Aware Perturbation Generation

This subsection describes Stage II, which addresses the *high-capacity* requirement through a shared expert pool and a feature-aware gating network.

1) *Shared Expert Generators*: The pool contains K expert generators $\{G_1, \dots, G_K\}$. Each expert takes a random noise vector $z \in \mathbb{R}^{d_z}$ and produces a candidate perturbation:

$$\delta_k = G_k(z), \quad \delta_k \in \mathbb{R}^N, \quad (12)$$

where N is the fixed maximum burst-sequence length. As shown in Fig. 5, each expert generator is implemented as a fully connected network with ReLU activations and dropout layers. All experts share the same architecture but have independent parameters, encouraging specialization in different perturbation patterns. Multiple experts expand the perturbation space beyond a single generator, while the shared pool avoids the linear parameter growth of per-class generators.

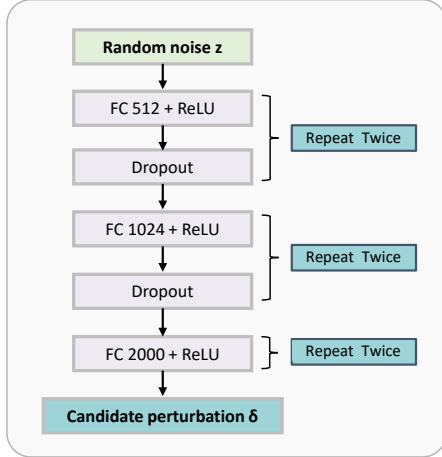


Fig. 5. The generator structure.

2) *Feature-Aware Gating Network*: Different websites exhibit different burst lengths, direction-switch frequencies, and local structures, making a uniform perturbation suboptimal. The gating network $M(\cdot)$ adaptively weights experts for each input.

Given representation $r = \Phi(bs)$, the gating network outputs a weight vector via temperature-scaled Softmax:

$$w = \text{Softmax}\left(\frac{M(r)}{\tau_s}\right), \quad w \in \mathbb{R}^K, \quad \sum_{k=1}^K w_k = 1, \quad (13)$$

where τ_s controls sparsity: smaller τ_s concentrates weight on fewer experts; larger τ_s yields a smoother mixture. In our implementation, τ_s is initialized to 0.5 and annealed after each epoch during training. This encourages broad expert

exploration in early training and gradually sharpens the routing distribution toward more suitable experts in later epochs. The candidate perturbations are then aggregated according to the expert-routing weights, producing the final perturbation:

$$\delta = \sum_{k=1}^K w_k \cdot G_k(z). \quad (14)$$

The gating network maps traffic representations into a low-dimensional *expert-combination space* interpretable as a defense-rule space: nearby gating vectors imply similar perturbation styles. This geometric structure is critical for Stage III, where we aggregate gating vectors from a few support samples into a class-level routing prototype without retraining.

Moreover, sharing a single expert pool across all classes decouples model capacity from the number of monitored websites, eliminating the linear parameter growth that plagues per-class defenses (addressing the scalability limitation).

D. Few-Shot Routing-Prototype Induction

This subsection presents Stage III, addressing *few-shot adaptability*. The key idea is to estimate the expert-combination preference of an unseen class by averaging the gating vectors of a small support set, then reuse the trained expert pool to produce a static defense rule.

Given the support set of a novel class y' , $S_{y'} = \{bs_1, \dots, bs_{n_s}\}$, the system computes per-sample representations and routing vectors:

$$r_i = \Phi(bs_i), \quad w_i = \text{Softmax}\left(\frac{M(r_i)}{\tau_s}\right). \quad (15)$$

The class-level routing prototype is obtained by mean aggregation:

$$\bar{w}_{y'} = \frac{1}{|S_{y'}|} \sum_{bs_i \in S_{y'}} w_i. \quad (16)$$

The defense rule for y' is then:

$$\delta_{y'} = \sum_{k=1}^K \bar{w}_{y',k} \cdot G_k(z). \quad (17)$$

Why mean aggregation works. The supervised contrastive pretraining (Stage I) produces tight, low-variance clusters per class in feature space. Because the gating network is a smooth mapping from feature space to expert-weight space, same-class samples concentrate in a compact region of the simplex Δ^{K-1} . The mean of these points therefore provides a low-bias estimate of the class centroid even for small $|S_{y'}|$, analogous to the prototypical-network principle [20] but operating in the routing-weight space rather than the classification embedding. This is empirically validated in Sec. V-E, where 5–10 samples suffice for stable adaptation.

The entire procedure involves only forward passes and arithmetic averaging—no backpropagation—reducing the marginal cost of onboarding a new class to sample collection and a single inference pass. Thus, for any previously unseen website, a defense rule can be deployed immediately after collecting a handful of traces, closing the protection gap without any retraining.

Algorithm 1 summarizes the complete induction process. Given a small support set from an unseen class, the algorithm first computes the representation and routing vector for each support trace, then averages these routing vectors into a class-level prototype. This prototype is finally used to weight the shared expert outputs, producing a static perturbation vector that can be directly deployed as the defense rule for the new class.

Algorithm 1 Few-Shot Routing-Prototype Induction

Require: Support set $S_{y'}$; feature extractor Φ ; gating network M ; expert generators $\{G_k\}_{k=1}^K$; temperature τ_s ; noise z

Ensure: Perturbation vector $\delta_{y'}$

```

1: for all  $bs_i \in S_{y'}$  do
2:    $r_i \leftarrow \Phi(bs_i)$ 
3:    $w_i \leftarrow \text{Softmax}(M(r_i)/\tau_s)$ 
4: end for
5:  $\bar{w}_{y'} \leftarrow \frac{1}{|S_{y'}|} \sum_{bs_i \in S_{y'}} w_i$ 
6: for  $k = 1$  to  $K$  do
7:    $\delta_k \leftarrow G_k(z)$ 
8: end for
9:  $\delta_{y'} \leftarrow \sum_{k=1}^K \bar{w}_{y',k} \cdot \delta_k$ 
10: return  $\delta_{y'}$ 

```

E. Loss Function Design

The training objective jointly optimizes four terms addressing defense effectiveness, bandwidth efficiency, perturbation stealthiness, and MoE training stability:

$$L_{\text{total}} = \alpha L_{\text{cf}} + \beta L_{\text{ds}} + \lambda L_{\text{bc}} + \mu L_{\text{bal}}, \quad (18)$$

where α , β , λ , and μ are weighting coefficients.

1) *Classification Misdirection Loss*: This term minimizes the substitute model's confidence on the true class. Let bs_{adv} be the defended trace, $\Gamma(\cdot)$ the substitute model, and y the true label:

$$L_{\text{cf}} = \max(\Gamma(bs_{\text{adv}})_y, 0). \quad (19)$$

During training, Γ and Φ are frozen; gradients flow only through M , and experts $\{G_k\}$.

Substitute model and transferability. Because the true attacker's architecture is unknown at defense time, we train with a substitute model Γ and rely on adversarial transferability—a standard practice in WF defenses [13], [14], [12]. We empirically validate transferability in Sec. V-D by evaluating against three distinct, independently trained attack architectures (AWF, DF, VarCNN).

2) *Perturbation Dispersion Loss*: The ℓ_1 norm constrains total dummy-packet volume but not its spatial distribution [13], [14]. Unconstrained optimization may concentrate perturbations on a few gradient-sensitive bursts, creating conspicuous traffic spikes. We introduce a dispersion penalty that exponentially penalizes per-burst insertions exceeding a threshold κ :

$$L_{\text{ds}} = \sum_{j=1}^m p^j, \quad p^j = \begin{cases} e^{\delta^j - \kappa} - 1, & \delta^j > \kappa, \\ 0, & \delta^j \leq \kappa, \end{cases} \quad (20)$$

where δ^j is the number of dummy packets at the j -th burst. The exponential growth strongly discourages heavy local injection while imposing no penalty on moderate insertions. Together with the bandwidth loss, this term controls both the *total volume* and the *spatial distribution* of perturbations.

3) *Bandwidth Overhead Loss*: To bound communication cost, we penalize relative overhead only when it exceeds a budget τ :

$$L_{\text{bc}} = \max\left(0, \frac{\|\delta\|_1}{\|bs\|_1} - \tau\right). \quad (21)$$

4) *Expert-Balancing Regularization*: Without explicit balancing, the gating network may collapse onto a small expert subset, underutilizing the shared pool. We regularize the variance of per-expert average utilization within each mini-batch:

$$L_{\text{bal}} = \text{Var}_{k=1}^K \left(\frac{1}{B} \sum_{i=1}^B w_{i,k} \right), \quad (22)$$

where $w_{i,k}$ is the gating weight for expert k on sample i . This encourages balanced expert utilization and prevents expert collapse, preserving the full representational capacity of the multi-expert architecture.

Algorithm 2 summarizes the overall training procedure of FAME. The feature extractor is first pretrained with supervised contrastive learning to obtain a class-discriminative representation space. The gating network and shared expert generators are then jointly optimized with the frozen substitute model, where the total loss combines classification misdirection, bandwidth control, perturbation dispersion, and expert-balancing regularization. The trained feature extractor, gating network, and expert pool are subsequently reused for both known-class defense and few-shot routing-prototype induction.

F. Burst-Level Perturbation Injection

In practice, the defender can only insert packets; deleting or reordering real packets would break Tor semantics. Furthermore, bidirectional injection requires client-server coordination. We therefore restrict injection to the *upstream* direction.

The raw perturbation δ is post-processed into a deployable rule. For the j -th burst:

$$\bar{\delta}^j = \text{round}(\text{ReLU}(\delta^j)) \cdot \eta^j, \quad (23)$$

where the direction mask η^j is:

$$\eta^j = \begin{cases} 1, & d^j = +1 \text{ (upstream)}, \\ 0, & d^j = -1 \text{ (downstream)}. \end{cases} \quad (24)$$

ReLU enforces non-negativity, rounding yields integer packet counts, and the mask restricts injection to upstream bursts. The resulting vector $\bar{\delta}$ is directly executable by the PT plugin or P4 switch (Sec. IV).

G. Complexity and Scalability Analysis

We analyze the computational and storage complexity of FAME to highlight its scalability advantages.

Parameter count. Let $|\theta_\Phi|$, $|\theta_M|$, and $|\theta_G|$ denote the parameter counts of the feature extractor, gating network, and a

Algorithm 2 Training Procedure of FAME

Require: Training set (BS, Y) ; feature extractor Φ ; gating network M ; experts $\{G_k\}_{k=1}^K$; substitute model Γ ; epochs T

Ensure: Trained Φ , M , $\{G_k\}_{k=1}^K$

```
1: Pretrain  $\Phi$  via supervised contrastive learning (Sec. III-B)
2: Initialize  $M$  and  $\{G_k\}_{k=1}^K$ 
3: for  $t = 1$  to  $T$  do
4:   Sample mini-batch  $\{(bs_i, y_i)\}_{i=1}^B$  from  $(BS, Y)$ 
5:   for all  $(bs_i, y_i)$  in the mini-batch do
6:      $r_i \leftarrow \Phi(bs_i)$ 
7:      $w_i \leftarrow \text{Softmax}(M(r_i)/\tau_s)$ 
8:     Sample  $z_i \sim \mathcal{N}(0, I)$ 
9:     for  $k = 1$  to  $K$  do
10:       $\delta_{i,k} \leftarrow G_k(z_i)$ 
11:    end for
12:     $\delta_i \leftarrow \sum_{k=1}^K w_{i,k} \delta_{i,k}$ 
13:     $bs_{\text{adv}}^i \leftarrow bs_i + \delta_i$ 
14:  end for
15:  Compute  $L_{\text{cf}}, L_{\text{ds}}, L_{\text{bc}}, L_{\text{bal}}$ 
16:   $L_{\text{total}} \leftarrow \alpha L_{\text{cf}} + \beta L_{\text{ds}} + \lambda L_{\text{bc}} + \mu L_{\text{bal}}$ 
17:  Update  $M, \{G_k\}_{k=1}^K$  via backprop (freeze  $\Phi, \Gamma$ )
18: end for
19: return  $\Phi, M, \{G_k\}_{k=1}^K$ 
```

single expert generator, respectively. The total parameter count is $|\theta_\Phi| + |\theta_M| + K \cdot |\theta_G|$, which is *independent of the number of website classes* C . A per-class design requires $C \cdot |\theta_G|$ parameters, growing linearly with C . For our configurations ($K \leq 10$, C up to 200), FAME uses at least an order of magnitude fewer generator parameters.

Training cost. Training is performed once on $\mathcal{Y}_{\text{base}}$. The per-epoch cost is $\mathcal{O}(B \cdot K)$ forward passes through experts, where B is the batch size. Because K is small (5–10), overhead over a single-generator baseline is modest.

Few-shot adaptation cost. Adapting to a novel class requires $|S_{y'}|$ forward passes through Φ and M , one pass per expert, and averaging—all $\mathcal{O}(|S_{y'}| \cdot K)$ with no gradient computation, completing in milliseconds.

Deployment footprint. The static defense rule $\bar{\delta}$ for each class is a single integer vector of length N , directly loaded into the PT plugin or P4 Burst Table.

IV. DEPLOYMENT

To validate practical deployability, we implement FAME in two complementary settings: a software-based Tor PT plugin for endpoint users, and a hardware-based data-plane extension on a Tofino P4 programmable switch. Both execute the same burst-level dummy-packet injection rules.

A. Tor PT Bridge Deployment

We deploy FAME as a PT plugin [22] installed on a remote server configured as a Tor Bridge node. Traffic between the client and the Bridge is modified according to precomputed

perturbation rules by inserting dummy packets into designated bursts.

At runtime, the client maintains a `BurstCounter` that synchronously tracks the current burst index. When the main goroutine reads a new batch of data from the buffer for forwarding, the counter increments and the perturbation logic is triggered: the system consults the perturbation vector to determine whether the current burst requires dummy-packet insertion. If so, the specified number of dummy packets is generated and forwarded to the send goroutine. The Bridge side mirrors this logic.

B. P4 Switch Data Plane Deployment

The burst-level perturbation mechanism is realized directly in the data plane of a Tofino P4 programmable switch. The control plane precomputes perturbation rules and installs them into an on-chip *Burst Table* stored in high-speed SRAM, where each entry maps a burst index within a Tor flow to the corresponding dummy-packet count.

During forwarding, the ingress pipeline maintains per-flow state to track the current burst index for each direction of a Tor connection; the index increments whenever a new burst is detected. When the index matches a Burst Table entry, the ingress stage annotates packets with metadata specifying the required dummy count. This metadata propagates to the egress stage, where the hardware packet-replication engine produces dummy packets and transmits them immediately after the original burst.

C. Deployment Summary

Both paradigms enable fine-grained, burst-level traffic shaping without modifying the Tor browser’s core architecture. Because dummy packets are appended at the end of each burst, latency overhead is negligible. We quantify this in Sec. V-I1 (PT plugin) and Sec. V-I2 (P4 switch), showing that the PT plugin’s overhead is statistically indistinguishable from natural network jitter, and the P4 implementation adds only ~ 300 ns of end-to-end latency. Hardware resource consumption on the P4 switch is analyzed in Sec. V-I3.

V. EVALUATION

In this section, we validate the effectiveness of FAME through the following experiments:

- (1) **Expert-Number Sensitivity:** We study how the number of experts K in the shared generator pool affects defense performance. Results show that increasing K from 1 to a moderate value (5–8) consistently improves defense, with diminishing returns thereafter, guiding the configurations for all subsequent experiments (Sec. V-C).
- (2) **Known-Class Closed-World Defense:** We compare FAME with five representative WF defenses on the AWF100 and DF datasets against three deep-learning attacks (AWF, DF, VarCNN), and further examine cross-model transferability across these attacks. In closed-world settings, FAME achieves the lowest attack accuracy across

all dataset–attack combinations while maintaining bandwidth overhead around 5%, with additional evidence of transferability across attack architectures (Sec. V-D).

- (3) **Few-Shot Adaptation to Newly Emerging Classes:** We evaluate whether FAME can defend website classes unseen during training when provided with only a small support set and no parameter updates. Results show that FAME substantially outperforms both rule-based and adversarial baselines, with 5–10 support samples sufficient for stable adaptation (Sec. V-E).
- (4) **Perturbation Dispersion Analysis:** We quantify the spatial distribution of generated perturbations using the coefficient of variation. Results confirm that FAME produces significantly more dispersed dummy-packet distributions than existing adversarial defenses, reducing the risk of detectable local traffic anomalies (Sec. V-F).
- (5) **Ablation Study:** We consolidate evidence from targeted experimental variants—Random Gate and Random Routing—and perturbation-distribution analysis to examine the contribution of each design component. Results show that feature-aware gating and routing-prototype induction improve defense effectiveness, while the dispersion-aware objective helps reduce burst-level perturbation concentration (Sec. V-G).
- (6) **Open-World Evaluation:** We evaluate FAME under a highly imbalanced open-world setting (monitored-to-unmonitored ratio of 1:100). Results show that FAME pushes the attacker’s AUC close to 0.5, rendering defended monitored traffic nearly indistinguishable from unmonitored traffic (Sec. V-H).
- (7) **Deployment Overhead:** We implement the burst-level injection mechanism on a Tor PT plugin and a Tofino P4 switch. Results show negligible impact on page loading time, approximately 300 ns additional per-packet latency on the P4 switch, and limited additional hardware resource consumption with no extra TCAM usage, confirming practical deployability (Sec. V-I).

A. Datasets and Experimental Setup

a) Datasets: We use two WF datasets: AWF [2] and DF [3]. For known-class experiments, we use AWF100 and DF. AWF additionally provides monitored-class sets at different scales, enabling the emerging-class scenario: AWF100 serves as the training set, and classes in AWF200 that do not overlap with AWF100 serve as unseen test classes. All raw traces are converted to burst sequences, uniformly truncated or padded to a fixed length.

b) Attack models: We use three deep-learning WF attack models spanning diverse architectures: AWF (standard CNN) [2], DF (deep CNN) [3], and VarCNN (residual CNN) [4]. Evaluating against architecturally diverse classifiers tests whether generated perturbations transfer across attack designs (Sec. III-E).

c) Baseline methods: We compare with five representative WF defenses: ALERT [13], AWA [14], FRONT [9], DFD [10], and Walkie-Talkie [8]. For known-class experiments, each baseline generates defense rules for the corresponding monitored classes according to its original perturbation mechanism, except for FRONT. Since FRONT relies on timestamped packet traces and schedules dummy packets on the real time axis, we implement a FRONT-inspired burst-level approximation for our burst-sequence setting. This variant preserves FRONT’s core ideas of prefix-focused dummy insertion and per-trace randomization by treating valid burst indices as a normalized pseudo-time axis. For few-shot experiments, adversarial baselines (ALERT and AWA) lack a native mechanism for generating defense rules for unseen classes without retraining; we therefore adopt random known-class rule transfer as a no-retraining baseline.

d) Implementation details: All traffic is represented as burst sequences with maximum length 2000. The model is implemented in TensorFlow with batch size 512 and 40 training epochs. The target bandwidth-overhead budget is 5%. Due to stochastic training effects and method-specific perturbation mechanisms, measured test-time overhead may deviate slightly from this target. The implementation of FAME and the reproduction code for the compared baselines are available at: <https://anonymous.4open.science/r/Code-of-FAME-80AC>.

B. Evaluation Metrics

a) Attack accuracy (ACC): The fraction of defended traces correctly classified by the attacker:

$$\text{ACC} = \frac{1}{N} \sum_{i=1}^N \mathbb{I}(\hat{y}_i = y_i). \quad (25)$$

Lower ACC indicates stronger defense.

b) Macro-averaged F1 score (AvgF1): Per-class recognition balance:

$$\text{AvgF1} = \frac{1}{C} \sum_{i=1}^C \frac{2 \cdot \text{PPV}_i \cdot \text{TPR}_i}{\text{PPV}_i + \text{TPR}_i + \epsilon}, \quad (26)$$

where PPV_i and TPR_i are precision and recall of class i .

c) Bandwidth overhead (BW): Additional communication cost from dummy-packet injection:

$$\text{BW} = \frac{1}{N} \sum_{i=1}^N \frac{\|\delta_i\|_1}{\|bs_i\|_1}. \quad (27)$$

d) Coefficient of variation (CV): Perturbation concentration across burst positions:

$$\text{CV} = \frac{\sigma(\delta)}{\mu(\delta)}, \quad (28)$$

where $\sigma(\delta)$ and $\mu(\delta)$ are the standard deviation and mean of the perturbation vector. Lower CV indicates more evenly distributed dummy packets.

TABLE I
KNOWN-CLASS CLOSED-WORLD DEFENSE COMPARISON. BW DENOTES BANDWIDTH OVERHEAD (%). LOWER AvgF1 AND ACC INDICATE STRONGER DEFENSE. **BOLD** MARKS THE BEST RESULT PER COLUMN.

Dataset	Defense Method	AWF			DF			VarCNN		
		AvgF1	ACC	BW	AvgF1	ACC	BW	AvgF1	ACC	BW
DF	Origin	0.9048	0.9047	–	0.9617	0.9614	–	0.9629	0.9627	–
	FRONT	0.5307	0.5184	5.17%	0.5314	0.5184	5.17%	0.5210	0.5063	5.17%
	DFD	0.6551	0.6534	10.61%	0.6487	0.6497	10.61%	0.5834	0.5935	10.61%
	Walkie-Talkie	0.4346	0.4380	28.97%	0.4355	0.4417	28.97%	0.3802	0.3887	28.97%
	ALERT	0.4015	0.4558	11.54%	0.5415	0.5865	10.80%	0.5633	0.6016	7.93%
	AWA	0.6790	0.7121	12.79%	0.7674	0.7925	6.87%	0.8462	0.8618	4.98%
	Random Gate	0.1334	0.1402	6.62%	0.1458	0.1586	5.54%	0.1637	0.1738	5.38%
	Ours	0.0721	0.0824	5.90%	0.0618	0.0749	5.14%	0.0842	0.1022	5.33%
AWF100	Origin	0.9335	0.9337	–	0.9824	0.9824	–	0.9876	0.9876	–
	FRONT	0.3359	0.3151	5.60%	0.3716	0.3476	5.60%	0.4214	0.3960	5.60%
	DFD	0.3091	0.3364	6.98%	0.1751	0.1976	6.98%	0.2966	0.3309	6.98%
	Walkie-Talkie	0.3756	0.3765	35.45%	0.3419	0.3505	35.45%	0.3428	0.3535	35.45%
	ALERT	0.3142	0.3747	7.77%	0.4123	0.4751	6.26%	0.4501	0.5248	4.88%
	AWA	0.6475	0.6813	5.89%	0.2432	0.2610	6.95%	0.4995	0.5379	6.59%
	Random Gate	0.0965	0.1025	5.15%	0.0762	0.0847	5.08%	0.1023	0.1124	5.41%
	Ours	0.0473	0.0484	4.92%	0.0444	0.0474	4.92%	0.0615	0.0649	5.02%

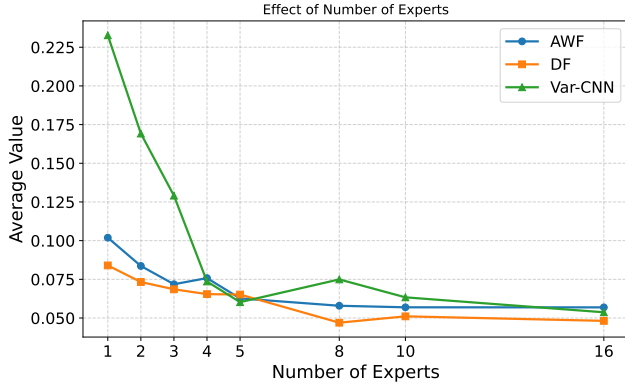


Fig. 6. Impact of the number of experts on defense effectiveness (AWF100). Each curve reports average attack accuracy over three seeds.

C. Expert-Number Sensitivity

The number of experts K controls the capacity of the shared generator pool. We analyze how defense performance varies with K on AWF100, setting $K \in \{1, 2, 3, 4, 5, 8, 10, 16\}$ with three random seeds per configuration.

As shown in Fig. 6, increasing K from 1 to a moderate value (5–8) consistently reduces attack accuracy, confirming that a larger perturbation space improves defense. Further increases yield only marginal gains and occasional fluctuations, indicating diminishing returns. We select K at the observed saturation point for each dataset–attack combination (Table II).

TABLE II
SELECTED EXPERT-NUMBER CONFIGURATIONS.

Dataset	DF	AWF	VarCNN
DF	5	8	5
AWF100	8	8	5
AWF200	10	10	8

D. Known-Class Closed-World Defense

Before evaluating adaptation to unseen classes, we verify that FAME is effective in the conventional known-class closed-world setting. We compare with FRONT, DFD, Walkie-Talkie, ALERT, and AWA on datasets DF and AWF100. We also include a **Random Gate** variant that uses the same shared expert pool but replaces learned gating weights with randomly sampled weights, isolating the contribution of the learned routing mechanism.

Table I summarizes the results. Without defense, all three attacks achieve high accuracy on both datasets, confirming the vulnerability of undefended Tor traffic. Under the low-bandwidth regime, FAME consistently achieves the lowest AvgF1 and ACC across all dataset–attack combinations. On AWF100, it reduces ACC to 0.0484, 0.0474, and 0.0649 against AWF, DF, and VarCNN, respectively, while maintaining bandwidth overhead around 5%.

Since FAME is optimized with a substitute model, We further examine cross-model transferability by training FAME with one substitute model and evaluating it against the other target attacks. Table III reports the results, where rows denote

TABLE III

TRANSFERABILITY ACROSS ATTACK MODELS. ROWS DENOTE SUBSTITUTE MODELS USED FOR TRAINING THE DEFENSE, AND COLUMNS DENOTE TARGET ATTACK MODELS USED FOR EVALUATION. VALUES ARE ACC; LOWER IS BETTER.

Dataset	Substitute	AWF	DF	VarCNN
AWF100	AWF	0.0484	0.1714	0.2285
	DF	0.1303	0.0474	0.2093
	VarCNN	0.1270	0.1377	0.0649
DF	AWF	0.0824	0.1716	0.2406
	DF	0.2165	0.0749	0.0973
	VarCNN	0.2019	0.1240	0.1022

substitute models and columns denote target models. The diagonal entries correspond to same-model evaluation, while off-diagonal entries measure transferability. Although same-model optimization achieves the strongest defense, FAME remains effective under cross-model settings: on AWF100, all transfer ACC values are no higher than 0.2285, and on DF they remain no higher than 0.2406. These results indicate that the generated perturbations are not tied to a single classifier architecture, but can transfer across AWF, DF, and VarCNN to some extent.

The comparison with Random Gate isolates the gating mechanism’s contribution. On the DF dataset, Random Gate obtains ACC of 0.1402/0.1586/0.1738, which FAME further reduces to 0.0824/0.0749/0.1022. On the AWF100 dataset, ACC drops from 0.1025/0.0847/0.1124 to 0.0484/0.0474/0.0649. This confirms that the performance gain is not solely from the shared expert pool; the learned gating network is essential for selecting effective expert combinations tailored to each traffic pattern.

E. Few-Shot Adaptation to Newly Emerging Classes

To evaluate rapid adaptation to unseen classes, we construct an emerging-class scenario from AWF100 and AWF200. The defense model is trained exclusively on AWF100; classes in AWF200 that do not overlap with AWF100 serve as unseen test classes.

For rule-based baselines (FRONT, DFD, Walkie-Talkie), their class-agnostic rules are applied directly to unseen-class traffic. For adversarial baselines (ALERT, AWA), generating class-specific perturbations for new classes would require retraining—the very limitation we address. We adopt random known-class rule transfer: a rule from a randomly selected AWF100 class is applied to the new class.

We report three FAME variants: **Random Routing** replaces the routing prototype with random expert weights; **Ours-FS** is the proposed few-shot method (5 support samples, no parameter updates); **Ours-Full** trains on complete AWF200 data as an upper bound.

a) Support-set size sensitivity: We vary the support-set size from 1 to 100 per unseen class. As shown in Fig. 7, performance improves from 1 to approximately 5 samples—most notably under VarCNN, where ACC drops from 0.1461

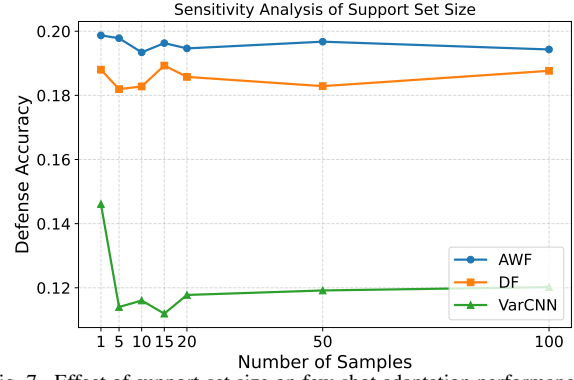


Fig. 7. Effect of support-set size on few-shot adaptation performance (unseen AWF200 classes). ACC is reported against three attack models.

(1-shot) to 0.1140 (5-shot). Beyond 5–10 samples, ACC stabilizes, confirming that a small support set suffices for reliable prototype estimation.

b) Comparison with baselines: Table IV reports the results. To ensure rigor, attack models are retrained on AWF200, assuming the attacker can learn the expanded class space while the defense remains trained only on AWF100. Without defense, the three attacks achieve ACC of 0.9239, 0.9820, and 0.9875. Rule-based defenses reduce accuracy to some extent but remain limited at low overhead; Walkie-Talkie achieves lower ACC at the cost of 36.86% bandwidth—far exceeding the low-overhead target.

Adversarial baselines show a clear disadvantage in this unseen-class setting. Under random rule transfer, ALERT obtains ACC of 0.5105/0.5877/0.6005 and AWA obtains 0.7070/0.2842/0.6263. In contrast, **Ours-FS** reduces ACC to 0.1819/0.1978/0.1140 at comparable bandwidth, demonstrating that routing-prototype induction generates substantially more effective defense rules than reusing perturbations from known classes.

c) Random Routing vs. Ours-FS: Random Routing already benefits from the shared expert pool (ACC: 0.2331/0.2430/0.2004). By replacing random weights with a support-set-induced prototype, Ours-FS further reduces ACC to 0.1819/0.1978/0.1140. This confirms that support samples provide class-specific routing information guiding expert combination, rather than the gain coming merely from multi-expert mixing.

d) Gap to full training: Ours-Full achieves ACC of 0.0611/0.0493/0.0263—lower than Ours-FS, as expected since it is trained with full AWF200 data. Nevertheless, Ours-FS achieves substantial defense without any retraining. Ours-Full further shows the capacity of the shared-expert architecture: when trained on the expanded AWF200 monitored set, the same framework achieves strong defense performance by moderately increasing the number of experts, without relying on per-class generators.

F. Perturbation Dispersion Analysis

To evaluate whether FAME mitigates over-concentrated injection, we analyze perturbation vectors generated by ad-

TABLE IV

FEW-SHOT ADAPTATION ON NEWLY EMERGING CLASSES. DEFENSE TRAINED ON AWF100, EVALUATED ON NON-OVERLAPPING AWF200 CLASSES.
BOLD: BEST NO-RETRAINING RESULT; UNDERLINE: FULL-TRAINING REFERENCE.

Defense Method	AWF			DF			VarCNN		
	AvgF1	ACC	BW	AvgF1	ACC	BW	AvgF1	ACC	BW
Origin	0.4871	0.9239	–	0.5553	0.9820	–	0.5864	0.9875	–
FRONT	0.2419	0.4164	5.03%	0.2218	0.3792	5.03%	0.2422	0.3935	5.03%
DFD	0.2337	0.4207	7.29%	0.1227	0.2296	7.29%	0.1391	0.2607	7.29%
Walkie-Talkie	0.1833	0.3054	36.86%	0.1562	0.2611	36.86%	0.1718	0.2876	36.86%
ALERT	0.3058	0.5105	6.71%	0.3402	0.5877	6.42%	0.3489	0.6005	4.66%
AWA	0.3959	0.7070	5.81%	0.1227	0.2842	7.21%	0.3573	0.6263	3.86%
Random Routing	0.1414	0.2331	5.03%	0.1360	0.2430	4.80%	0.1218	0.2004	5.00%
Ours-FS	0.0912	0.1819	5.35%	0.1172	0.1978	5.14%	0.0620	0.1140	5.62%
Ours-Full	<u>0.0578</u>	<u>0.0611</u>	4.94%	<u>0.0412</u>	<u>0.0493</u>	4.93%	<u>0.0198</u>	<u>0.0263</u>	4.93%

versarial defenses on AWF100. We report Avg CV (average coefficient of variation across classes) and Max CV (worst-case class).

TABLE V

PERTURBATION CONCENTRATION ANALYSIS ON AWF100. LOWER CV VALUES INDICATE MORE DISPERSED PERTURBATIONS.

Attack Model	ALERT	AWA	Ours
	Avg / Max CV	Avg / Max CV	Avg / Max CV
AWF	3.0034 / 13.3599	5.5467 / 18.3559	2.8131 / 8.5923
DF	3.2098 / 16.9214	0.9597 / 14.8318	2.6392 / 4.4401
VarCNN	4.1918 / 14.8792	4.3325 / 16.0312	2.8222 / 5.3080

As shown in Table V, FAME achieves the lowest Max CV under all three attack models, reducing it from 13.36/18.36 (ALERT/AWA) to 8.59 under AWF, from 16.92/14.83 to 4.44 under DF, and from 14.88/16.03 to 5.31 under VarCNN. This demonstrates that the dispersion loss effectively suppresses worst-case burst-level concentration and prevents extreme local spikes. For Avg CV, FAME also achieves the lowest values under AWF and VarCNN. These results confirm that FAME produces more spatially dispersed perturbations and reduces the risk of detectable local anomalies.

G. Ablation Study

Rather than reporting a full-factorial ablation table, we use targeted variants and perturbation-distribution analysis to examine key components. Random Gate and Random Routing isolate feature-aware gating and routing-prototype induction, respectively. CV analysis provides distribution-level evidence for the dispersion-aware objective, but is not a direct loss ablation.

a) Multi-expert capacity (Sec. V-C): Increasing K from 1 to a moderate value consistently reduces attack accuracy (Fig. 6), confirming that the multi-expert pool enlarges the perturbation space. Diminishing returns beyond $K \approx 8$ indicate that capacity saturates.

b) Feature-aware gating router (Sec. V-D): The Random Gate variant in Table I uses the same expert pool with random weights. FAME consistently outperforms it (e.g., ACC 0.0484 vs. 0.1025 on AWF100/AWF), demonstrating that learned gating provides meaningful traffic-dependent expert selection.

c) Routing-prototype induction (Sec. V-E): Random Routing in Table IV replaces the support-set-induced prototype with random weights. Ours-FS consistently outperforms it (e.g., ACC 0.1140 vs. 0.2004 under VarCNN), confirming that support samples provide class-specific routing information essential for unseen-class adaptation.

d) Perturbation dispersion objective (Sec. V-F): Table V shows that FAME achieves lower Max CV than ALERT and AWA under all three attack models, indicating less severe worst-case burst-level concentration. This result is consistent with the design of the dispersion loss, which penalizes excessive per-burst insertion. Meanwhile, FAME also achieves the lowest ACC in Table I, suggesting that reducing local perturbation concentration does not compromise defense effectiveness.

In summary, the shared expert pool provides capacity for diverse traffic patterns; feature-aware gating is necessary for effective expert combination; routing-prototype induction enables unseen-class adaptation without retraining; and the dispersion loss helps mitigate concentrated perturbation patterns.

H. Open-World Evaluation

In the open-world setting, the attacker performs binary detection—determining whether a trace belongs to a monitored class—rather than multi-class classification. We evaluate under a 1:100 monitored-to-unmonitored ratio to reflect realistic class imbalance.

The ROC curve plots the true positive rate against the false positive rate across decision thresholds. AUC near 1.0 indicates strong detection; AUC near 0.5 indicates indistinguishability from random guessing. An effective defense should push AUC toward 0.5.

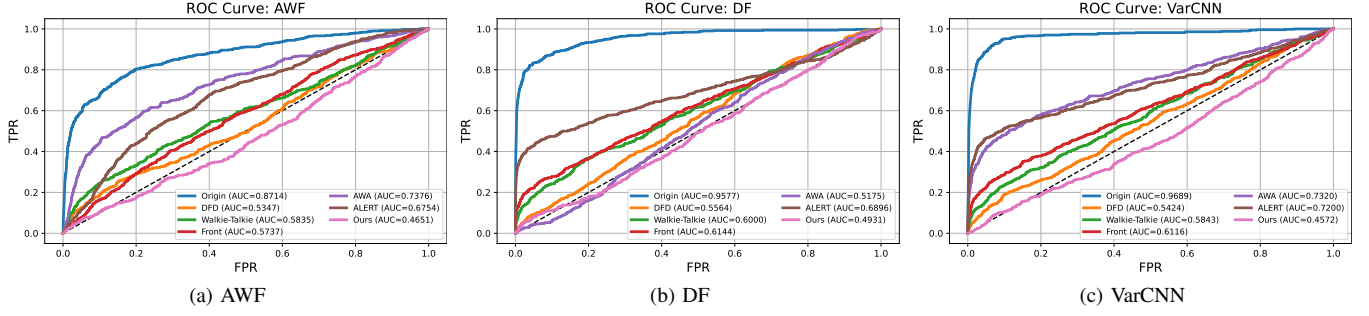


Fig. 8. ROC curves under the open-world setting (monitored : unmonitored = 1 : 100) on AWF100.

TABLE VI
PT PLUGIN DEPLOYMENT PERFORMANCE. “OURS” DENOTES FAME.

BW	Total Time (s)			Start Transfer (s)		
	Orig.	Ours	Diff.	Orig.	Ours	Diff.
2M	2.784	2.896	+0.112	1.908	1.913	+0.005
5M	3.147	2.933	-0.214	2.140	1.937	-0.203
10M	3.053	2.953	-0.100	1.992	1.950	-0.042

Fig. 8 shows the ROC curves on AWF100. Under AWF and VarCNN, FAME achieves AUC values of 0.4651 and 0.4572, respectively—closest to 0.5 among all defenses, indicating that defended traffic is nearly indistinguishable from unmonitored traffic. Under DF, the AUC differs from 0.5 by only 0.0069.

We note that AUC values slightly below 0.5 (e.g., 0.4572) indicate that the classifier’s predictions are weakly *anti-correlated* with the true label—a phenomenon that arises when defended monitored traffic is statistically shifted to resemble unmonitored traffic more than actual unmonitored traffic does, effectively “overshooting” the disguise. From a defender’s perspective, this is benign: it means the attacker cannot gain useful discrimination even by inverting its decision rule, since the deviation from 0.5 is within statistical noise. These results confirm that FAME provides strong open-world protection under highly imbalanced conditions.

I. Deployment Experiments

We evaluate practical deployment overhead in two settings.

1) *PT Plugin Deployment*: We integrate FAME into a custom PT plugin and measure page loading times for Wikipedia across three bandwidth tiers (2 Mbps, 5 Mbps, 10 Mbps). For each configuration, we conduct 50 independent trials, fixing the guard node (our bridge) and exit node to mitigate network fluctuations.

As shown in Table VI, latency overhead is negligible in both time-to-first-byte and total duration. At 5 Mbps and 10 Mbps, defended traffic is even slightly faster than the baseline—attributable to inherent network jitter exceeding the defense overhead. These results confirm that FAME can be deployed transparently without degrading browsing performance.

2) *P4 Data Plane Deployment*: We implement the burst-level injection mechanism on an Intel Tofino switch. Two Ubuntu servers with Intel E810 100GbE NICs are connected to the switch. We measure per-packet end-to-end latency under

four bandwidth settings (1, 2.5, 5, and 10 Mbps) for both plain forwarding and FAME-enabled forwarding, using hardware timestamps across 10 runs of 5,000 packets each.

As shown in Fig. 9, enabling FAME introduces approximately 300 ns of additional end-to-end latency, primarily due to queuing of generated dummy packets. Processing latency remains at the nanosecond scale, confirming negligible overhead.

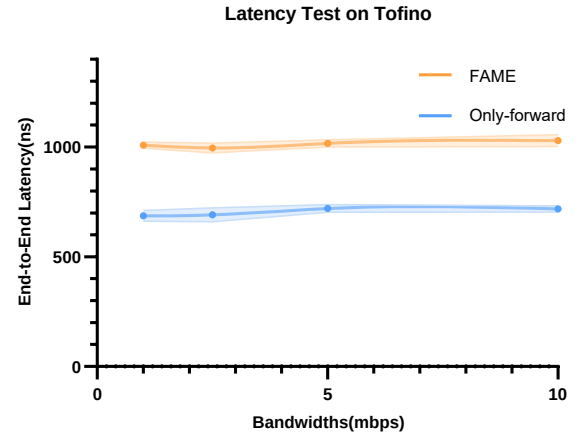


Fig. 9. Per-packet end-to-end latency on the Tofino P4 switch under four bandwidth settings.

TABLE VII
ADDING FAME TO FORWARDING-ONLY P4 PROGRAM
TNA_SIMPLE_SWITCH.P4

Computational	eMatch xBar	tMatch xBar	Gateway	VLIW	Hash bits
Forwarding-only usage + FAME	17.30% +4.58%	14.29% 0.00%	8.93% +4.46%	16.96% +2.24%	16.41% +4.98%
Memory	SRAM	TCAM	Map RAM	Table ID	Stash
Forwarding-only usage + FAME	18.39% +7.86%	15.48% 0.00%	2.68% +10.12%	24.11% +6.25%	14.29% +2.67%
PHV	8bits used	16bits used	32bits used	Overall bits used	
Forwarding-only usage + FAME	20.31% +4.69%	87.50% 0.00%	96.88% +3.12%	70.98% +2.23%	

3) *Hardware Resource Footprint on P4 Switches*: To understand the impact of deploying FAME on existing networks, we design two experiments to evaluate the additional overhead introduced by the system. First, to quantify FAME’s consumption of on-chip memory and computational resources on programmable switches, we integrate FAME into a feature-rich and logically complex forwarding program, `tna_simple_switch.p4`, which support most of protocol in real network, like Ethernet, IPv4/IPv6, VLAN, UDP, TCP, GTP-U, and SRv6, and compare the switch resource utilization before and after the integration. The results are summarized in Table VII. We observe that, compared to the baseline, our design does not incur any additional overhead on the tMatch crossbar or the scarce TCAM resources, which important for regular forwarding algorithm like `ipv4_lpm`, and introduces only marginal increases in other resources. These results indicate that FAME can be seamlessly embedded into complex existing switch programs without compromising the extensibility of switch functionality.

VI. RELATED WORK

A. WF Attacks

Early WF attacks relied on manual feature engineering: Panchenko et al. [23] combined statistical features with SVMs, and Wang et al. [24] extracted over 3,000 features for k-NN, achieving over 90% closed-world accuracy. With deep learning, neural-network-based attacks became dominant.

Rimmer et al. [2] proposed AWF, showing that CNNs achieve 96% accuracy. Sirinam et al. [3] introduced DF, a deeper CNN reaching 98%+ accuracy that has become the de facto evaluation baseline. Bhat et al. [4] proposed VarCNN with residual connections for stronger representation learning. Together, AWF, DF, and VarCNN span standard, deep, and residual architectures, motivating their use as our attack models.

Recent work has explored multi-tab browsing [6], [25], few-shot attacks [26], and Transformer-based temporal modeling [27]. However, timing features remain fragile under network fluctuations [28]; direction-based models like DF remain the standard benchmark for defense robustness evaluation.

B. WF Defenses

1) *Obfuscation-Based Defenses*: Obfuscation-based defenses disrupt traffic patterns through padding, timing perturbation, or communication-model changes. WTF-PAD [7] inserts statistically distributed dummy packets between bursts for zero-delay protection. DFD [10] dynamically tracks burst lengths and proportionally injects dummy packets. FRONT [9] targets the feature-rich initial phase by injecting during the early handshake. Walkie-Talkie [8] enforces half-duplex communication with supersequence loading.

While exhibiting some robustness against traditional classifiers, these defenses degrade substantially against deep-learning attacks (e.g., WTF-PAD still yields 77.24% accuracy under DF). Several also incur high bandwidth overhead. We

include FRONT, DFD, and Walkie-Talkie as representative obfuscation baselines.

2) *Adversarial-Example-Based Defenses*: Adversarial defenses seek a better effectiveness–overhead trade-off [29], [11], [28], [13], [14], [30]. WF-GAN [29] and Mockingbird [11] generate adversarial traces but require complete traces before perturbation, precluding real-time deployment. BLANKET [12] generates perturbations from random noise without prior knowledge of website content; however, it uses a single generator without class-aware routing and cannot adapt to unseen classes at deployment time. Minipatch [31] applies patch-like perturbations via simulated annealing. AWA [14] and ALERT [13] adopt burst-sequence representations: AWA trains paired generators to mimic each other’s traffic, while ALERT directs samples toward other classes via adversarial training.

Despite promising results, these methods share two fundamental limitations that FAME directly addresses. First, each requires a separate generator or template per class, causing linear scaling of training cost and model complexity. Second, none provides a mechanism for adapting to classes unseen during training—when new classes emerge, the entire retrain–redeploy pipeline must be repeated. FAME resolves both through its shared MoE architecture and routing-prototype induction.

A further gap is perturbation stealthiness. Existing adversarial defenses concentrate injection on gradient-sensitive bursts, creating conspicuous statistical footprints. No prior work explicitly incorporates a stealth-aware objective. Our perturbation-dispersion loss (Sec. V-F) fills this gap by regularizing the spatial distribution of dummy packets.

VII. CONCLUSION

We presented FAME, a few-shot adaptive perturbation generation framework for WF defense based on the Mixture-of-Experts architecture. FAME addresses three fundamental limitations of existing adversarial defenses—scalability, adaptability, and stealthiness—through a shared expert pool with feature-aware gating, a routing-prototype induction mechanism, and a perturbation-dispersion loss, respectively.

Experiments demonstrate that FAME achieves the lowest attack accuracy in closed-world settings at ~5% bandwidth overhead, adapts to unseen classes with 5–10 support samples and no retraining, produces substantially more dispersed perturbations than prior adversarial defenses, and renders defended traffic indistinguishable from unmonitored traffic ($AUC \approx 0.5$) in the open-world setting. Deployment on a Tor PT plugin and a Tofino P4 switch confirms practical feasibility with negligible latency overhead.

The results suggest that decoupling perturbation capacity from class count—and enabling continual adaptation via prototype induction—offers a viable path toward deployable, long-lived WF defenses that can keep pace with the evolving web landscape.

ETHICAL CONSIDERATIONS

This research proposes FAME, a defensive framework designed to enhance user privacy in anonymous communication systems by mitigating Website Fingerprinting (WF) attacks. We have considered the ethical implications of this work following the principles of responsible security research.

Risk vs. Benefit. The primary benefit of FAME is to strengthen privacy protection for users who may access the Internet under surveillance or censorship. The proposed defense only inserts dummy packets and does not modify, delay, drop, or inspect real user payloads. We acknowledge that traffic perturbation techniques introduce additional bandwidth overhead and may affect shared network resources if used excessively. To limit this risk, FAME is designed under a strict low-overhead constraint, and our experiments evaluate its bandwidth cost together with defense effectiveness. Our PT plugin and P4 switch experiments further evaluate deployment overhead, including page-loading time, per-packet latency, and switch resource footprint, to ensure that the proposed mechanism remains practical under controlled deployment conditions.

Data Privacy and Subjects. This study does not involve human subjects, and we do not collect, intercept, or analyze real users' private communications. All attack and defense evaluations are conducted on publicly available academic WF datasets, including AWF and DF, which contain traffic traces for research purposes and do not include communication payloads or personally identifiable information. Our analysis focuses only on side-channel traffic features such as packet directions, burst structures, and dummy-packet distributions. The deployment experiments are conducted in controlled settings using our own PT plugin and P4 switch testbed, and no third-party user traffic is monitored or modified.

Dual Use and Responsible Research. We recognize that traffic obfuscation techniques may have dual-use implications: while they protect user privacy, they may also make some forms of traffic analysis more difficult. In this work, FAME is designed and evaluated strictly as a defensive mechanism against Website Fingerprinting attacks on anonymous communication users. The proposed method only generates burst-level dummy-packet insertion rules to reduce website-identifying side-channel patterns; it does not provide capabilities for traffic interception, payload inspection, exploitation, command-and-control communication, or user deanonymization. We also do not disclose any new exploitable vulnerability in Tor or network infrastructure. By presenting FAME and its evaluation, our goal is to support the development of more robust privacy-preserving communication systems and provide a transparent basis for future defensive research.

REFERENCES

- [1] R. Dingledine, N. Mathewson, and P. Syverson, “Tor: The second-generation onion router,” 2004.
- [2] V. Rimmer, D. Preuveneers, M. Juarez, T. Van Goethem, and W. Joosen, “Automated website fingerprinting through deep learning,” *arXiv preprint arXiv:1708.06376*, 2017.
- [3] P. Sirinam, M. Imani, M. Juarez, and M. Wright, “Deep fingerprinting: Undermining website fingerprinting defenses with deep learning,” in *Proceedings of the 2018 ACM SIGSAC conference on computer and communications security*, 2018, pp. 1928–1943.
- [4] S. Bhat, D. Lu, A. Kwon, and S. Devadas, “Var-cnn: A data-efficient website fingerprinting attack based on deep learning,” *arXiv preprint arXiv:1802.10215*, 2018.
- [5] Y. Huang, W. Qiao, D. Wang, M. Shen, D. Zhao, L. Li, S. Cui, B. Jiang, Z. Lu, and B. Liu, “Forge: A robust multi-tab website fingerprinting attack via blind source separation,” in *Proceedings of the ACM Web Conference 2026*, 2026, pp. 3018–3029.
- [6] S. Wang, X. Hu, G. Cheng, R. Li, and H. Wu, “Stmwf: Multi-tab website fingerprinting via spatial-temporal sequence analysis,” *IEEE Transactions on Information Forensics and Security*, 2026.
- [7] M. Juárez, M. Imani, M. Perry, C. Diaz, and M. Wright, “Wtf-pad: toward an efficient website fingerprinting defense for tor,” *CoRR*, abs/1512.00524, 2015.
- [8] T. Wang and I. Goldberg, “{Walkie-Talkie}: An efficient defense against passive website fingerprinting attacks,” in *26th USENIX Security Symposium (USENIX Security 17)*, 2017, pp. 1375–1390.
- [9] J. Gong and T. Wang, “Zero-delay lightweight defenses against website fingerprinting,” in *29th USENIX Security Symposium (USENIX Security 20)*, 2020, pp. 717–734.
- [10] A. Abusnaina, R. Jang, A. Khormali, D. Nyang, and D. Mohaisen, “Dfd: Adversarial learning-based approach to defend against website fingerprinting,” in *IEEE INFOCOM 2020-IEEE Conference on Computer Communications*. IEEE, 2020, pp. 2459–2468.
- [11] M. S. Rahman, M. Imani, N. Mathews, and M. Wright, “Mockingbird: Defending against deep-learning-based website fingerprinting attacks with adversarial traces,” *IEEE Transactions on Information Forensics and Security*, vol. 16, pp. 1594–1609, 2020.
- [12] M. Nasr, A. Bahramali, and A. Houmansadr, “Defeating {DNN-Based} traffic analysis systems in {Real-Time} with blind adversarial perturbations,” in *30th USENIX Security Symposium (USENIX Security 21)*, 2021, pp. 2705–2722.
- [13] L. Qiao, B. Wu, H. Li, C. Gao, W. Yuan, and X. Luo, “Trace-agnostic and adversarial training-resilient website fingerprinting defense,” in *IEEE INFOCOM 2024-IEEE Conference on Computer Communications*. IEEE, 2024, pp. 211–220.
- [14] A. M. Sadeghzadeh, B. Tajali, and R. Jalili, “Awa: Adversarial website adaptation,” *IEEE Transactions on Information Forensics and Security*, vol. 16, pp. 3109–3122, 2021.
- [15] R. Xie, K. Xie, P. Zhao, J. He, X. Zeng, J. Wen, Y. Xie, W. Liang, and G. Xie, “Gapdis: Gradient-assisted perturbation design via sequence editing for website fingerprinting defense,” in *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security*, 2025, pp. 3012–3026.
- [16] L. Qiao, B. Wu, S. Yin, H. Li, W. Yuan, and X. Luo, “Resisting dnn-based website fingerprinting attacks enhanced by adversarial training,” *IEEE Transactions on Information Forensics and Security*, vol. 18, pp. 5375–5386, 2023.
- [17] Y. Xie, J. Feng, W. Huang, Y. Zhang, X. Sun, X. Chen, and X. Luo, “Contrastive fingerprinting: A novel website fingerprinting attack over few-shot traces,” in *Proceedings of the ACM Web Conference 2024*, 2024, pp. 1203–1214.
- [18] X. Deng, Y. Zhang, Q. Li, Z. Liu, Y. Wang, and K. Xu, “Enhancing website fingerprinting attacks against traffic drift,” *Network and Distributed System Security (NDSS) Symposium*, 2026.
- [19] N. Shazeer, A. Mirhoseini, K. Maziarz, A. Davis, Q. Le, G. Hinton, and J. Dean, “Outrageously large neural networks: The sparsely-gated mixture-of-experts layer,” *arXiv preprint arXiv:1701.06538*, 2017.
- [20] J. Snell, K. Swersky, and R. Zemel, “Prototypical networks for few-shot learning,” *Advances in neural information processing systems*, vol. 30, 2017.
- [21] P. Khosla, P. Teterwak, C. Wang, A. Sarna, Y. Tian, P. Isola, A. Maschinot, C. Liu, and D. Krishnan, “Supervised contrastive learning,” *Advances in neural information processing systems*, vol. 33, pp. 18 661–18 673, 2020.
- [22] torproject.org, “Tor pluggable transport,” <https://tb-manual.torproject.org/circumvention/>.
- [23] A. Panchenko, L. Niessen, A. Zinnen, and T. Engel, “Website fingerprinting in onion routing based anonymization networks,” in *Proceedings of the 10th annual ACM workshop on Privacy in the electronic society*, 2011, pp. 103–114.
- [24] T. Wang, X. Cai, R. Nithyanand, R. Johnson, and I. Goldberg, “Effective attacks and provable defenses for website fingerprinting,” in *23rd USENIX Security Symposium (USENIX Security 14)*, 2014, pp. 143–157.
- [25] Y. Yuan, W. Zou, and G. Cheng, “Mw3f: Improved multi-tab website fingerprinting attacks with transformer-based feature fusion,” *Journal of Network and Computer Applications*, vol. 236, p. 104125, 2025.
- [26] C. Luo, W. Tang, Q. Wang, and D. Zheng, “Few-shot website fingerprinting with distribution calibration,” *IEEE Transactions on Dependable and Secure Computing*, vol. 22, no. 1, pp. 632–648, 2024.
- [27] Q. Zhou, L. Wang, H. Zhu, T. Lu, and V. S. Sheng, “Wf-transformer: Learning temporal features for accurate anonymous traffic identification by using transformer networks,” *IEEE Transactions on Information Forensics and Security*, vol. 19, pp. 30–43, 2023.
- [28] N. Mathews, J. K. Holland, S. E. Oh, M. S. Rahman, N. Hopper, and M. Wright, “Sok: A critical evaluation of efficient website fingerprinting defenses,” in *2023 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2023, pp. 969–986.
- [29] C. Hou, G. Gou, J. Shi, P. Fu, and G. Xiong, “Wf-gan: Fighting back against website fingerprinting attack using adversarial learning,” in *2020 IEEE Symposium on Computers and Communications (ISCC)*. IEEE, 2020, pp. 1–7.
- [30] J. Gong, W. Zhang, C. Zhang, and T. Wang, “Surakav: Generating realistic traces for a strong website fingerprinting defense,” in *2022 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2022, pp. 1558–1573.
- [31] D. Li, Y. Zhu, M. Chen, and J. Wang, “Minipatch: Undermining dnn-based website fingerprinting with adversarial patches,” *IEEE Transactions on Information Forensics and Security*, vol. 17, pp. 2437–2451, 2022.